Algorithm Engineering **Summer 2026**

Dr. Phillip Keldenich
Dr. Dominik Krupke

Submission due: 04.06.2026 13:15

Homework 4

Due to the excursion week, this is essentially half of a regular exercise sheet, giving half the points for roughly half the work of a regular sheet. Work in teams of two. Homework is due every two weeks.

Submission format: Host your solution in a private Git repository on one of the following platforms and add Rouven as a collaborator: IBR GitLab (`kniep`), TU GitLab (`r.kniep`), GitHub (`rkniep`), or Codeberg (`rkniep`). If you want to use a different platform, please ask first.

Before the deadline, send the link to your submission commit by e-mail to `kniep@ibr.cs.tu-bs.de`.

Exercise 1 (Branch Variable Selection): (10 + 7 + 8 points)

The `exercise01` subdirectory for this exercise contains the same skeleton Branch & Bound implementation in Python for the Knapsack problem with mutual exclusivity constraints that was used in exercise 1 of sheet 3. For details on the problem, the existing code and its installation, refer to that homework assignment sheet.

If you struggled with sheet 3 and already submitted your solution, a version with a solution for sheet 3's tasks is available on request; of course, you can and should work with your own solution if you have solved the tasks of exercise 1. While your improvements in the last sheet should already have had a noticeable impact on solver performance, there still are some improvements to do and different approaches and parameters to evaluate.

Perform the following improvements in order; report the runtime and number of nodes visited reported by `mek-solve benchmark_150.json` after each step.

a) As a first step, we want to improve on our branch variable selection strategy by implementing *strong branching* near the root of the search tree. Given a fractional solution x^* with multiple fractional variables at some node of our branch & bound tree, strong branching evaluates some number k of candidate variables x_i by solving, at least partially, the LP relaxations resulting from branching x_i up and down (to $x_i = 1$ and $x_i = 0$ for our binary variables).

   For this purpose, the existing python code contains a method `_solve_and_reset_bounds` that takes two partial assignments, `old` and `new`. Here, `old` should be the partial assignment that is currently active in the LP solver, i.e., the one that the parent node was just solved with. The method returns the LP relaxation value resulting from fixing the partial assignment `new`, resetting the bounds in the LP solver to `old` before returning.

   As a tunable parameter, define a list $[k_0, k_1, \ldots, k_d], k_i \geq 2$ for some d; at a branch & bound node at depth j, if $j \leq d$, consider the up to k_j fractional variables with the

largest $\min(1 - x_i, x_i)$, i.e., distance to integrality, as candidates for branching, and evaluate the LP relaxation values E_- for the down branch and E_+ for the up branch; call `constraint_propagation` on the new partial assignments before handing them to `_solve_and_reset_bounds`. Implement the following approaches for selecting the branch variable: (i) $\min E_+ \cdot E_-$, (ii) $\min E_+ + E_-$, (iii) $\min \max(E_+, E_-)$ and (iv) $\min \min(E_+, E_-)$. At a branch & bound node at depth $j > d$, fall back to the distance-to-integrality metric instead.

Try several values for d (in the range 2–7) and $k_0, \ldots, k_d$ until you achieve a noticeable runtime improvement over the version without strong branching, and evaluate which of the approaches (i)-(iv) reports the least time spent in the LP solver.

b) Armed with strong branching, we want to prepare the implementation of reliability branching (a combination of pseudo-cost branching and strong branching). For this purpose, extend the `VariableStatistics` class to track the number of times the down branch and up branch of each variable has been evaluated, as well as tracking the pseudo-cost scores $\Psi_+(x_i), \Psi_-(x_i)$ for each variable, and the `update_statistics` method to update the statistics of the branch variable x_i each time we solved the LP relaxation of a child. Do not forget to also update the pseudo-cost scores whenever a branch is evaluated via strong branching.

For the update, introduce parameters $\rho \in \mathbb{N}$ and $\lambda \in [0, 1]$. If $\min(n_-(x_i), n_+(x_i)) < \rho$, where $n_-(x_i)$ and $n_+(x_i)$ are the number of times we have evaluated the down and up branch of x_i, we consider the scores of x_i to be unreliable.

Updating an unreliable score, which is initially 0, works as follows. Let x_i^p be the value of the branch variable x_i in the parent's LP relaxation, which has value p. We assume a down branch, i.e., $x_i = 0$ in the child's LP relaxation, which has value c; up branches work analogously with $\lceil x_i^p \rceil - x_i^p$ instead of $x_i^p - \lfloor x_i^p \rfloor$. We then first increment $n_-(x_i)$, followed by updating

$$\Psi_-(x_i) \leftarrow \frac{\Psi_-(x_i) \cdot (n_-(x_i) - 1) + \frac{p-c}{(x_i^p - \lfloor x_i^p \rfloor)}}{n_-(x_i)}.$$

For a reliable score, the update is

$$\Psi_-(x_i) \leftarrow \lambda \Psi_-(x_i) + (1 - \lambda) \frac{p - c}{(x_i^p - \lfloor x_i^p \rfloor)}.$$

Since this step should not change the search tree, you do not need to rerun your code or report its output for this step.

c) We now want to put our pseudo-cost scores to use and implement reliability branching. Define two more integer parameters, $d_r \geq d$ and k_r. These represent the maximum depth at which we ever consider performing strong branching and the corresponding number of candidates to explore. As in a), below depth d, we always use strong branching, although the best value for d may need to be changed.

For deeper nodes, there are the following cases:

- If all fractional variables have reliable pseudo-cost scores (both up and down branches are sufficiently explored), use the pseudo-cost scores to estimate E_-

and E_+ for each candidate; then proceed as in a), using one of the approaches (i)–(iv).

- Otherwise, if the node is at depth at most d_r, evaluate up to k_r of the most fractional unreliable variables to compute their E_+, E_- values; then use these values and the E_+, E_- values derived from the pseudo-costs for the reliable variables.
- Otherwise, at depth higher than d_r, let n_r be the number of reliable candidates and n_u be the number of unreliable candidate variables. Generate a random integer q (add a random number generator to the solver class with a fixed seed) from $[0, n_r + n_u - 1]$; if $q < n_r$, select the best reliable candidate according to their E_+ and E_- estimates; otherwise, branch on the unreliable variable with the largest distance to integrality.

Re-evaluate your approach for different values of d, d_r and k_r; which of the approaches (i)-(iv) performs best?